

PDE Top-30 Deep-Read Report

Stage 5: LLM-refined reproducibility analysis

OpenClaw / Ollie — subagent for Rick

2026-04-24

Contents

1	Executive Summary	2
2	Per-paper deep reads	4
3	Recommended 15-paper batch schedule	27
4	Conclusion	27

1. Executive Summary

This report distills Stage-5 deep reads of the top-30 PDE papers surviving Stage 4 signal-based scoring (all with *full PDFs* in hand — abstract-only papers excluded per Rick’s instructions). For each paper, we read sections from the extracted text (methods / experiments / conclusions / tail-references), verified advertised GitHub/GitLab URLs by HTTP check, and produced a structured reproduction plan.

Tier list

- **Priority A (start today, 11 papers):** Public, live code + clear benchmarks. Low friction, high confidence of quantitative replication.
- **Priority B (plan next 1–2 weeks, 10 papers):** Partial code, heavy dependencies, or tractable reimplementation.
- **Priority C (aspirational / deferred, 9 papers):** No public code or theory-focused; reimplementation cost dominates.

Top-10 revised ranking (by reproduction feasibility $\times$ scientific impact)

1. **FEM vs PINNs** (Grossmann 2023) — A, feasibility 10/10, 4h
2. **ML-accelerated CFD / jax-cfd** (Kochkov 2021) — A, 9/10, 6h, 1123 cites
3. **Dedalus** (Burns 2019) — A, 9/10, 4h, 485 cites
4. **Latent Spectral Models** (Wu 2023) — A, 9/10, 6h
5. **Walk on Stars** (Sawhney 2023) — A, 9/10, 5h
6. **Koopman Neural Operator** (Xiong 2023) — A, 8/10, 6h
7. **Laplace Neural Operator** (Cao 2023) — A, 8/10, 4h
8. **Fast Poisson spectral** (Fortunato/Townsend 2017) — A, 8/10, 4h
9. **Lightning Laplace/Helmholtz** (Gopal/Trefethen 2019) — A, 9/10, 3h
10. **Kinetic.jl** (Xiao 2021) — A, 9/10, 4h

Key changes from Stage-4 rank

- **Promoted to A:** Kochkov (up from 6), Dedalus (up from 7) — both have exemplary public code.
- **Demoted to C:** Kopanicakova 2023 (code “upon acceptance” only), Foucart 2022 (github.com/corbett5/amr_dl returns 404), Bessemoulin 2022 / Blömker 2013 / Gantner 2020 (all theory-only, no code).
- **Off-topic flag:** Xu 2022 “Poisson Flow Generative Models” is a generative ML paper that uses the Poisson equation as an *analogy*; it’s not really a PDE-solver paper. Retained as B-tier since code is excellent but scope is a mismatch.

Domain / method distribution

- ML-operator / PINN: 10 papers (~33%)
- Classical spectral / FD / FV: 8 papers
- Preconditioning / DD / multigrid: 4 papers
- Stochastic / kinetic / fractional: 5 papers
- Monte Carlo / grid-free: 2 papers
- AMR / adaptivity: 3 papers

A healthy 40–60 split between ML-forward and classical numerics, matching our strategic intent.

Recommended “start today” set

Spawn Wave 1 immediately (5 parallel subagents, ~ 8 h wallclock): FEM-vs-PINNs, Walk on Stars, Fast Poisson Spectral, Lightning Laplace, Kinetic.jl. See `top30_batch_schedule.md` for full 15-paper waves.

Cross-check against existing replications

A text search across `~/Dropbox/REPLICATE-PROJECT/` found *no* matching titles or DOIs among the existing ~ 35 subdirectories (which cover cosmology, materials, biology, NLP, etc.). All 30 PDE candidates are novel to our replication portfolio.

2. Per-paper deep reads

#1. Can physics-informed neural networks beat the finite element method?

Authors: T. G. Grossmann; U. J. Komorowska; J. Latz; C. Schönlieb • **Year:** 2023 • **Cites:** 188 • **DOI:** 10.1093/imamat/hxae011

Priority: A • **Revised score:** 19/20 • **Topic:** ML-operator/PINN

Problem. Systematically compare PINNs versus the finite element method (FEM) for solving a suite of linear and nonlinear PDEs. The question: can PINNs actually beat FEM in accuracy and runtime?

Approach. Solve 1D/2D/3D Poisson, 1D Allen-Cahn, and 1D/2D semilinear Schrödinger with both standard Galerkin FEM (FEniCS) and PINNs (PyTorch). Sweep PINN architectures (layers, nodes) and FEM mesh sizes. Measure relative error on independent test grid, solution-time and evaluation-time.

Key results.

- For all PDEs tested, FEM achieved same-or-better accuracy at 2–3 orders of magnitude lower cost than PINNs.
- PINNs were only competitive on pointwise evaluation cost after training, not on total time to solution.
- Larger PINN networks did not yield monotonic accuracy gains; width mattered more than depth.

Main contribution. Carefully-controlled empirical disproof of the common claim that PINNs outperform FEM for standard elliptic/parabolic PDEs.

Reproducibility. Code available: *yes*. URL: <https://github.com/TamaraGrossmann/FEM-vs-PINNs> (HTTP 200). Language: python (FEniCS + PyTorch). Completeness: 9/10. Data: Analytical/manufactured solutions, generated in-repo.

Benchmarks. Poisson 1D/2D/3D, Allen-Cahn 1D, Schrödinger 1D/2D (semilinear)

Reproduction plan (feasibility 10/10, ~4h, compute: 1 GPU):

1. Clone github.com/TamaraGrossmann/FEM-vs-PINNs and install FEniCS + PyTorch.
2. Run 1D Poisson FEM (their `fem_poisson1d.py`) across mesh refinements; record L2 relative errors vs DOF.
3. Train PINNs at specified architectures on 1D Poisson; record train time and L2 error.
4. Reproduce Pareto plot (Fig 2): FEM dominates PINN in time-accuracy tradeoff.
5. Repeat for Allen-Cahn 1D and confirm qualitative conclusion.
6. Optional: extend with 3D Poisson to reproduce the one regime (eval time) where PINN wins.

Expected fidelity: Quantitative match within ~10% on relative-error tables; qualitative reproduction of all Pareto figures.. **Known risks:** PINN training is stochastic; averaged over several seeds for stable numbers.; FEniCS version drift (legacy vs FEniCSx)..

Follow-on questions.

- Does the PINN-vs-FEM gap close at higher spatial dimensions (d5)?
- How do adaptive activation functions or Fourier feature embeddings change the Pareto?
- Is the conclusion robust under parametric PDE families where FEM must be resolved many times (surrogate regime)?
- What happens under irregular geometry where FEM meshing is itself expensive?
- Does causal PINN training alter the Allen-Cahn verdict?

Rationale for priority: *Perfect match to our needs: open code, FEniCS+PyTorch only, direct quantitative comparison. Keep top rank.*

#2. Solving High-Dimensional PDEs with Latent Spectral Models

Authors: Haixu Wu;Tengge Hu;Huakun Luo;Jianmin Wang;Mingsheng Long • **Year:** 2023 • **Cites:** 95 • **DOI:** 10.48550/arXiv.2301.12664

Priority: A • **Revised score:** 18/20 • **Topic:** ML-operator/PINN

Problem. Neural operators work in coordinate space and suffer curse-of-dimensionality for high-dim PDEs. Learn PDEs in a compact latent spectral space.

Approach. Latent Spectral Models (LSM) use hierarchical projection to a low-dim latent, then a 'neural spectral block' that approximates the operator with trainable basis functions. Theoretical guarantees via spectral approximation theory.

Key results.

- SOTA on 7 solid+fluid benchmarks (Elasticity, Plasticity, Navier-Stokes, Darcy, Airfoil, Pipe).
- Stable performance across resolutions $32 \times 32 \rightarrow 1024 \times 1024$ on Darcy.
- Consistent outperformance of FNO, U-FNO, WMT, Galerkin-Transformer baselines.

Main contribution. Operator learning in a learned latent spectral basis—decoupling representation from output grid.

Reproducibility. Code available: *yes*. URL: <https://github.com/thuml/Latent-Spectral-Models> (HTTP 200; also in [thuml/Neural-Solver-Library](#)). Language: python (PyTorch). Completeness: 10/10. Data: PDEBench / Takamoto benchmarks; dataset download links in repo README.

Benchmarks. Elasticity-P, Elasticity-G, Plasticity, Navier-Stokes (2D), Darcy, Airfoil, Pipe

Reproduction plan (feasibility 9/10, ~6h, compute: 1 GPU (A100 ideal, any modern GPU sufficient)):

- 1. Clone [thuml/Latent-Spectral-Models](#); install PyTorch per requirements.txt.
- 2. Download Darcy 2D dataset (small benchmark, fast iteration).
- 3. Train LSM-S on Darcy (should take ~1–2h on a single GPU); reproduce relative L2 near 0.0065 (paper value).
- 4. Train FNO baseline from the same repo on same data; compare to LSM.
- 5. Run resolution test across 64/128/256 on Darcy to replicate resolution-stability figure.
- 6. (Optional) Navier-Stokes 2D benchmark for a harder check.

Expected fidelity: Quantitative match within 5–10% on Darcy & NS-2D L2 errors.. **Known risks:** GPU memory at 1024x1024 resolution.; Dataset download bandwidth (10+ GB)..

Follow-on questions.

- How does LSM perform on 3D turbulence datasets (Johns Hopkins Turbulence Database)?
- Can the latent-spectral basis be made physics-aware (e.g., divergence-free)?
- What is the sample-efficiency curve vs. FNO?
- Does LSM generalize zero-shot across parametric PDE families?
- Can the approach be combined with multigrid for hybrid solver acceleration?

Rationale for priority: *Well-supported repo and benchmark datasets; strong baselines. Prime candidate.*

#3. Koopman neural operator as a mesh-free solver of non-linear partial differential equations

Authors: Wei Xiong;Xiaomeng Huang;Ziyang Zhang;Ruixuan Deng;Pei Sun;Yang Tian • **Year:** 2023 • **Cites:** 74 • **DOI:** 10.48550/arXiv.2301.10022

Priority: A • **Revised score:** 17/20 • **Topic:** ML-operator/PINN

Problem. Existing neural operators become inaccurate and less interpretable for long-term/non-linear PDE evolution.

Approach. Koopman Neural Operator (KNO) formulates the PDE solution as a non-linear dynamical system and approximates its Koopman operator (infinite-dim linear) acting on flow embeddings. Uses an FNO-like lifting but with a Koopman block.

Key results.

- Outperforms FNO on mesh-independent Burgers experiments and long-term prediction on 5 PDEs incl. 2D Navier-Stokes and Rayleigh-Bénard.
- Zero-shot prediction across untrained discretization granularities.
- Applied to real systems: global water vapor, western boundary currents.

Main contribution. First Koopman-theoretic neural operator with demonstrated long-horizon stability on turbulent PDEs.

Reproducibility. Code available: *yes*. URL: <https://github.com/Koopman-Laboratory/KoopmanLab> (HTTP 200). Language: python (PyTorch). Completeness: 9/10. Data: Benchmark datasets reused from FNO paper (Burgers, NS-2D) + Copernicus ocean data.

Benchmarks. Bateman-Burgers 1D, 2D Navier-Stokes, Rayleigh-Bénard, Shallow Water, Diffusion-reaction

Reproduction plan (feasibility 8/10, ~6h, compute: 1 GPU (A100 40GB used by authors)):

1. Clone Koopman-Laboratory/KoopmanLab; install via `pip install -e .`
2. Download FNO's Burgers dataset; run `python train_burgers.py` with KNO config.
3. Verify reported relative L2 ~ reported value (check paper Table 2).
4. Train FNO baseline for head-to-head.
5. Extend to NS-2D long-term (100+ steps) and check stability.
6. Zero-shot: train at `res=32`, evaluate at `res=64/128`.

Expected fidelity: Within 10-15% on L2; qualitative reproduction of Koopman-eigenvalue analysis.. **Known risks:** Long-term rollout stability is finicky.; Possible version drift in KoopmanLab package..

Follow-on questions.

- How does KNO compare against POD-DL-ROM on same benchmarks?
- Is the learned Koopman spectrum interpretable / linked to physical modes?
- Can KNO handle boundary changes (mesh-free in a stronger sense)?
- Performance on 3D turbulence?
- Hybrid KNO + classical time-integrator for stability guarantees?

Rationale for priority: *Strong code, clear benchmarks. Slightly trickier than LSM due to long-rollout stability.*

#4. Lifex-cfd: an Open-source Computational Fluid Dynamics Solver for Cardiovascular Applications

Authors: P. C. Africa;Ivan Fumagalli;M. Bucelli;Alberto Zingaro;L. Dede';A. Quarteroni • **Year:** 2023 • **Cites:** 61 • **DOI:** 10.48550/arXiv.2304.12032

Priority: B • **Revised score:** 16/20 • **Topic:** Fluid/NavierStokes

Problem. Open-source CFD solver specialized for cardiovascular simulations (blood flow in moving cardiac chambers + valves).

Approach. Finite-element incompressible Navier-Stokes solver on deal.II/PETSc. ALE for moving domains, VMS-LES for transitional turbulence, RIIS for immersed valves. SUPG-PSPG stabilization.

Key results.

- Verified ideal convergence orders on Beltrami benchmark.
- Near-ideal parallel speedup on Taylor-Green vortex.

- Patient-specific cases for idealized and real vasculature.

Main contribution. Production-quality, LGPL-v3 cardiovascular CFD with careful verification and parallel scalability.

Reproducibility. Code available: *yes*. URL: <https://gitlab.com/lifex/lifex-cfd> (HTTP 200) + Zenodo 10.5281/zenodo.7852088 for input data. Language: C++ (deal.II, PETSc, Trilinos). Completeness: 10/10. Data: Zenodo 7852088 - parameter files, meshes, auxiliary data.

Benchmarks. Beltrami flow, Taylor-Green vortex, patient-specific aorta/LV

Reproduction plan (feasibility 7/10, ~10h, compute: multi-CPU node; HPC for full cardiac test):

- 1. Build deal.II with MPI/PETSc/Trilinos; build lifex core, then lifex-cfd.
- 2. Download Zenodo 7852088; run Beltrami benchmark → verify convergence table.
- 3. Run Taylor-Green vortex on 32-core node; verify expected kinetic-energy dissipation curve.
- 4. (Optional) Run one patient-specific idealized aorta case from the provided data.

Expected fidelity: Quantitative match on convergence rates and kinetic-energy dissipation curves.. **Known risks:** deal.II build is non-trivial (1–3h compile).; Full cardiac cases may require HPC queue time..

Follow-on questions.

- Can the lifex-cfd Navier-Stokes solver be coupled with Windkessel BCs from the literature?
- Performance vs. OpenFOAM on the same Beltrami benchmark?
- Can ML turbulence models replace VMS-LES?
- GPU port via deal.II’s matrix-free infrastructure?
- Verification on pulsatile non-Newtonian rheology.

Rationale for priority: *Excellent reproducibility but heavy dependency stack (deal.II + Trilinos) inflates setup time. Great if we want a cardiovascular reference; not a ‘start today’ pick.*

#5. Enhancing training of physics-informed neural networks using domain-decomposition based preconditioning strategies

Authors: Alena Kopanicáková;Hardik Kothari;G. Karniadakis;Rolf Krause • **Year:** 2023 • **Cites:** 28 • **DOI:** 10.1137/23M1583375

Priority: C • **Revised score:** 13/20 • **Topic:** ML-operator/PINN

Problem. PINN training often converges slowly with L-BFGS; preconditioning is underexplored.

Approach. Introduce Schwarz-domain-decomposition-based nonlinear preconditioners (additive SPQN and multiplicative MPQN) that decompose network parameters layer-wise and apply L-BFGS within blocks.

Key results.

- Significant convergence acceleration vs vanilla L-BFGS/Adam on several PINN benchmarks.
- Additive preconditioner is embarrassingly parallel → new model-parallelism route for PINNs.
- Lower achieved relative error at fixed training budget.

Main contribution. Brings classical domain-decomposition preconditioning into nonlinear optimization for PINNs.

Reproducibility. Code available: *no*. URL: Promised upon acceptance (Bitbucket/alena_ referenced in text); SIAM J. Sci. Comput. 2023 version should have code appendix. Language: python (PyTorch per conventions of Karniadakis group). Completeness: 4/10. Data: Manufactured solutions for Burgers, Allen-Cahn, advection tests.

Benchmarks. Burgers, Allen-Cahn, advection-diffusion

Reproduction plan (feasibility 6/10, ~8h, compute: 1 GPU):

- 1. Implement layer-wise Schwarz block L-BFGS from scratch guided by Algorithm 1 in paper.
- 2. Reproduce Burgers PINN baseline with Adam and plain L-BFGS.
- 3. Add additive SPQN wrapper; verify convergence curves reported in Fig 5.
- 4. Measure final Erel and compare to paper Tables 3-5.

Expected fidelity: Qualitative reproduction of convergence acceleration; numerical match within 20% given stochasticity.. **Known risks:** No public code yet $\rightarrow$ full re-implementation required.; L-BFGS line search finicky in Schwarz setting..

Follow-on questions.

- Does the preconditioner help for stiff PINN losses (low-viscosity Burgers)?
- How does it compose with causal/self-adaptive PINN weighting?
- Scaling to 100-layer PINNs?
- Comparison to natural-gradient PINN variants?
- Theoretical convergence rate of the nonlinear Schwarz iteration.

Rationale for priority: *Strong idea, but code not public at time of writing — demotes from A to C until repo is released or full reimplement budget allotted.*

#6. Machine learning–accelerated computational fluid dynamics

Authors: Dmitrii Kochkov; Jamie A. Smith; Ayya Alieva; Qing Wang; M. Brenner; Stephan Hoyer • **Year:** 2021

• **Cites:** 1123 • **DOI:** 10.1073/pnas.2101784118

Priority: A • **Revised score:** 19/20 • **Topic:** ML-operator/PINN

Problem. DNS of Navier-Stokes is limited by smallest-scale resolution; LES relies on hand-tuned closures.

Approach. End-to-end differentiable CFD where a CNN corrects the convective flux on a coarse grid; trained against high-res DNS of 2D Kolmogorov flow with model-consistent rollouts (32 steps).

Key results.

- Matches DNS accuracy at 8–10 $\times$ coarser grid $\rightarrow$ ~40–80 $\times$ compute speedup.
- Stable long-horizon rollouts (tens of thousands of time steps).
- Outperforms LES closures at equivalent cost.

Main contribution. First large-scale demonstration of ML-accelerated CFD with DNS-fidelity on 2D turbulence.

Reproducibility. Code available: *yes*. URL: <https://github.com/google/jax-cfd> (HTTP 200). Language: python (JAX). Completeness: 10/10. Data: Datasets generated in-repo (seeded); also published via TFDS.

Benchmarks. 2D Kolmogorov flow Re=1000, 2D decaying turbulence

Reproduction plan (feasibility 9/10, ~6h, compute: 1 GPU (Spark or A100)):

- 1. pip install jax-cfd; run the notebook ‘notebooks/kolmogorov_flow.ipynb’.
- 2. Generate ground-truth DNS at 2048² (may need to reduce to 512² for time budget).
- 3. Train the learned correction model (default config).
- 4. Compare vorticity correlation vs baseline DNS at same coarse resolution.
- 5. Reproduce Fig 1 quantitative results: coarse-DNS vs ML-DNS vorticity correlations.

Expected fidelity: Quantitative match on vorticity correlation and energy spectra.. **Known risks:** DNS at 2048² is expensive; may need to cap at 1024².; JAX-cfd’s default training can take hours..

Follow-on questions.

- Does the same approach work for 3D turbulence?

- Transfer learning across Reynolds numbers?
- Hybrid with entropy-stable limiters for shocks?
- Training with less high-res data (sample efficiency)?
- Integrating this with weather model cores (NWP)?

Rationale for priority: *1123 citations, Google JAX-CFD repo is exemplary, data generation built in. Upgrade to co-#1 priority.*

#7. Dedalus: A flexible framework for numerical simulations with spectral methods

Authors: K. Burns;G. Vasil;J. Oishi;D. Lecoanet;B. Brown • **Year:** 2019 • **Cites:** 485 • **DOI:** 10.1103/PhysRevResearch.2.0
Priority: A • **Revised score:** 18/20 • **Topic:** Fluid/NavierStokes

Problem. Need a flexible framework to solve general PDEs spectrally in many geometries (Cartesian, spherical, annular).

Approach. Dedalus: object-oriented Python framework representing PDEs symbolically; uses modified sparse tau method with Chebyshev/Fourier bases; banded matrices via Dirichlet preconditioning. Parallel via MPI.

Key results.

- Demonstrated on Kelvin-Helmholtz, Rayleigh-Bénard, Kolmogorov flow, self-gravitating cylinders.
- Banded systems give $O(N)$ solves per timestep.
- Scalable to $O(10k)$ cores on ALCF/NERSC.

Main contribution. Widely-adopted (485 cites) production-grade spectral PDE framework.

Reproducibility. Code available: *yes*. URL: <https://github.com/DedalusProject/dedalus> (HTTP 200) + [DedalusProject/methods_paper_examples](#). Language: python (NumPy, MPI, h5py). Completeness: 10/10. Data: Example scripts in [methods_paper_examples](#) repo.

Benchmarks. Poisson 1D/2D, Helmholtz, Kelvin-Helmholtz 2D, Rayleigh-Bénard, reaction-diffusion

Reproduction plan (feasibility 9/10, ~4h, compute: CPU (multi-core); MPI for 2D+ cases):

1. `conda install -c conda-forge dedalus`.
2. Clone [DedalusProject/methods_paper_examples](#).
3. Run 1D Poisson example; verify spectral convergence.
4. Run 2D Kelvin-Helmholtz example; compare to published vorticity field.
5. Run Rayleigh-Bénard at $Ra=1e6$; check Nusselt number scaling.

Expected fidelity: Exact match on spectral convergence; qualitative on turbulent cases.. **Known risks:** Dedalus v3 changed API significantly; [methods_paper_examples](#) may target v2.; MPI build can be finicky..

Follow-on questions.

- Benchmark Dedalus vs Julia's ApproxFun + FourierFlows on same problems.
- GPU port via JAX or CUDA?
- Automatic-differentiation overlay for inverse problems?
- Use for dynamo simulations — replicate Vasil's ICC benchmark.
- Coupling with AMR for local resolution.

Rationale for priority: *Mature, heavily cited, easy install. Upgrade — excellent starting point.*

#8. Physics-informed neural networks for solving Reynolds-averaged Navier-Stokes equations

Authors: Hamidreza Eivazi; Mojtaba Tahani; P. Schlatter; R. Vinuesa • **Year:** 2021 • **Cites:** 404 • **DOI:** 10.1063/5.0095270

Priority: B • **Revised score:** 13/20 • **Topic:** Fluid/NavierStokes

Problem. Turbulence closure is hard; can PINNs learn RANS solutions using only boundary data?

Approach. Fully-connected PINN taking $(x,y) \rightarrow (U,V,P,u^2,v^2,uv)$. Loss = PDE residual of RANS + data match at boundaries (no interior closure assumption).

Key results.

- $\sim 1\%$ error on laminar Falkner-Skan.
- Good accuracy on ZPG/APG boundary layers and NACA-0012 airfoil without a turbulence model.
- Reynolds stresses recovered consistently.

Main contribution. Shows PINNs can bypass explicit turbulence closure if boundary Reynolds-stress data are provided.

Reproducibility. Code available: *no*. URL: None advertised; may be available upon request from authors. Language: python (TensorFlow per text). Completeness: 3/10. Data: DNS/LES reference data for ZPG/APG BL and NACA (public DNS databases).

Benchmarks. Falkner-Skan, ZPG BL, APG BL, NACA-0012

Reproduction plan (feasibility 6/10, ~ 8 h, compute: 1 GPU):

1. Set up a 5-output PINN in DeepXDE with RANS residual.
2. Generate Falkner-Skan analytical solution; train; expect $\sim 1\%$ error.
3. Use Jiménez DNS for ZPG BL as boundary data; train.
4. Compare predicted U,V,P,u^2,v^2,uv against DNS at interior stations.

Expected fidelity: Qualitative (shapes of U,V profiles); quantitative L2 within $\sim 10\%$ of paper.. **Known risks:** No public code $\rightarrow$ reimplementations; PINN for turbulent cases is notoriously sensitive to loss-weight scheduling..

Follow-on questions.

- Does it extend to 3D separated flows?
- Comparison against data-driven SA or k - ω closures?
- Uncertainty quantification on the recovered Reynolds stresses.
- Integration with real PIV data instead of DNS.
- Does causal/time-marching PINN improve on this steady formulation?

Rationale for priority: *Clear, impactful paper but no public code lowers feasibility; still a B-tier candidate given DeepXDE reimplementations is tractable.*

#9. Learning in Modal Space: Solving Time-Dependent Stochastic PDEs Using Physics-Informed Neural Networks

Authors: Dongkun Zhang; Ling Guo; G. Karniadakis • **Year:** 2019 • **Cites:** 274 • **DOI:** 10.1137/19m1260141

Priority: C • **Revised score:** 11/20 • **Topic:** ML-operator/PINN

Problem. Long-time integration of stochastic PDEs with arbitrary initial data — classical DO/BO methods have singular covariance and eigenvalue-crossing issues.

Approach. NN-DO and NN-BO: parameterize modes and coefficients by neural networks; enforce DO/BO constraints implicitly in the loss (no explicit time derivatives of modes).

Key results.

- Works when original DO fails (deterministic IC case).
- RMSE for BO random coefficients $\sim 0.05\text{--}0.29$ at early/late times.
- Handles long-time integration via subdomain-in-time decomposition.

Main contribution. Trainable DO/BO decomposition avoiding the pathological assumptions of classical versions.

Reproducibility. Code available: *no*. URL: **None** advertised; possibly available from Karniadakis group on request. Language: python (TensorFlow 1.x, typical of 2019 Karniadakis). Completeness: 3/10. Data: Manufactured SPDE problems (linear stochastic advection, stochastic diffusion-reaction, etc.).

Benchmarks. Stochastic advection 1D, Stochastic diffusion-reaction 1D, Stochastic Burgers

Reproduction plan (feasibility 5/10, $\sim 10\text{h}$, compute: 1 GPU):

- 1. Reimplement the DO/BO loss with NN parameterization (algorithms 1-2 in paper).
- 2. Benchmark on stochastic advection; match RMSE ~ 0.05 after 1000 iterations.
- 3. Extend to diffusion-reaction; match Table 5 random-coefficient RMSE.

Expected fidelity: Qualitative match; quantitative within $2\times$ of reported RMSE.. **Known risks:** No code; 2019 paper $\rightarrow$ reimplementation.; Subdomain time-splitting detail is fragile..

Follow-on questions.

- Extension to stochastic Navier-Stokes 2D?
- Adaptive basis dimension during training?
- Comparison to stochastic Galerkin + PC expansion.
- Inverse problems under stochastic forcing.
- How much training data does the inverse setup require?

Rationale for priority: *Niche but no public code; demote to C.*

#10. Poisson Flow Generative Models

Authors: Yilun Xu; Ziming Liu; M. Tegmark; T. Jaakkola • **Year:** 2022 • **Cites:** 115 • **DOI:** 10.48550/arXiv.2209.11178

Priority: B • **Revised score:** 14/20 • **Topic:** Elliptic

Problem. Normalizing flows / SDE-based generative models have slow sampling and mode-collapse issues.

Approach. Interpret data as charges in $z=0$ plane of a $(d+1)$ -dim space; the gradient of the Poisson equation solution induces a deterministic flow that maps data $\rightarrow$ uniform hemisphere. Backward ODE gives sampling.

Key results.

- Inception 9.68, FID 2.35 on CIFAR-10 (SOTA at time among normalizing flows).
- 10-20 $\times$ faster sampling than VP/VE-SDE baselines at equal FID.
- Robust to network choice and step size.

Main contribution. A PDE-physics-inspired generative model with fewer sampling steps and good FID.

Reproducibility. Code available: *yes*. URL: https://github.com/Newbeeer/poisson_flow (HTTP 200). Language: python (PyTorch). Completeness: 10/10. Data: CIFAR-10, CelebA (public).

Benchmarks. CIFAR-10 FID/IS, CelebA 64x64, LSUN bedroom

Reproduction plan (feasibility 9/10, $\sim 10\text{h}$, compute: 1 GPU (A100 ideal); training 1-2 days):

- 1. Clone Newbeeer/poisson_flow.
- 2. Use provided CIFAR-10 config; train.
- 3. Evaluate FID/IS using their eval script.

- 4. Reproduce sampling-speedup table (Table in paper).

Expected fidelity: FID within ~ 0.2 of published 2.35.. **Known risks:** Training takes ~ 1 day.; FID calculation depends on reference stats; ensure matching CIFAR-10 stats file..

Follow-on questions.

- How does PFGM compare to later PFGM++ and diffusion-transformer models?
- Does the Poisson analogy extend to discrete data (text)?
- Connection to score matching and Wasserstein flows.
- Ablate the hemisphere geometry — is it necessary?
- Large-scale text-to-image applications.

Rationale for priority: *Not really a PDE-solver paper (it's a generative model using Poisson as analogy). Good code but off-topic for PDE replication; B tier at best for 'PDE' bucket.*

#11. LNO: Laplace Neural Operator for Solving Differential Equations

Authors: Qianying Cao;S. Goswami;G. Karniadakis • **Year:** 2023 • **Cites:** 56 • **DOI:** 10.48550/arXiv.2303.10528
Priority: A • **Revised score:** 16/20 • **Topic:** ML-operator/PINN

Problem. FNO handles only periodic signals and cannot represent non-periodic transient responses well.

Approach. Laplace Neural Operator (LNO): replace Fourier transform block with Laplace transform (pole-residue representation). Natural handling of transients via poles.

Key results.

- LNO with 1 layer beats FNO with 4 layers on 3 ODEs (Duffing, driven pendulum, Lorenz) and 3 PDEs (Euler-Bernoulli beam, diffusion, reaction-diffusion).
- Exponential convergence claimed for linear problems.
- Better transient-response capture in undamped cases.

Main contribution. Laplace-transform-based neural operator handling non-periodic and transient signals.

Reproducibility. Code available: *yes*. URL: <https://github.com/qianyingcao/Laplace-Neural-Operator> (HTTP 200). Language: python (PyTorch). Completeness: 9/10. Data: Generated in-repo via SciPy ODE solvers and analytical solutions.

Benchmarks. Duffing, driven pendulum, Lorenz, Euler-Bernoulli beam, diffusion, reaction-diffusion

Reproduction plan (feasibility 8/10, ~ 4 h, compute: 1 GPU):

- 1. Clone qianyingcao/Laplace-Neural-Operator.
- 2. Run Duffing training; verify relative L2 $\downarrow$ FNO baseline.
- 3. Run reaction-diffusion; verify Table 2 values.
- 4. Sweep # poles; produce the convergence-vs-poles plot.

Expected fidelity: Quantitative match on Table 2 within 10%.. **Known risks:** Complex-valued arithmetic in pole-residue layer can have numerical issues..

Follow-on questions.

- Extension to 2D PDEs on non-periodic domains?
- Can the learned poles be interpreted physically?
- Compare with wavelet/DeepONet operators.
- Efficiency vs. FNO at equal parameter count.
- Transfer learning across physical parameters.

Rationale for priority: *Clean scope, public code, lightweight benchmarks. Solid A pick.*

#12. Walk on Stars: A Grid-Free Monte Carlo Method for PDEs with Neumann Boundary Conditions

Authors: Rohan Sawhney;Bailey Miller;Ioannis Gkioulekas;Keenan Crane • **Year:** 2023 • **Cites:** 48 • **DOI:** 10.1145/3592398

Priority: A • **Revised score:** 17/20 • **Topic:** Elliptic

Problem. Walk-on-spheres Monte Carlo solves only Dirichlet Laplace problems; no Neumann BCs.

Approach. Walk on Stars (WoSt) samples on star-shaped regions whose boundaries include Neumann-boundary arcs; sampling via visibility silhouette. Solves linear elliptic PDEs with mixed BCs without meshing.

Key results.

- First grid-free MC method for Neumann conditions.
- Works on arbitrary geometries (non-convex) and extreme geometric complexity.
- Progressive output (like ray tracing) → useful for graphics.

Main contribution. Fundamentally extends WoS to the full mixed-BC regime with practical BVH-based implementation.

Reproducibility. Code available: *yes*. URL: <https://github.com/GeometryCollective/wost-simple> (HTTP 200). Language: C++ (with Python bindings); also simple Python reference. Completeness: 9/10. Data: Example scenes in repo.

Benchmarks. 2D Laplace mixed-BC, Poisson with Neumann, heat transfer (toaster-bread example)

Reproduction plan (feasibility 9/10, ~5h, compute: CPU (multi-thread)):

- 1. Clone wost-simple; build (CMake + C++17).
- 2. Run 2D Laplace demo with mixed Dirichlet/Neumann.
- 3. Verify pointwise convergence rate $N^{-1/2}$.
- 4. Compare against FEniCS reference solution on same geometry.

Expected fidelity: Quantitative MC convergence rate match; pointwise values within 1 of reference.. **Known risks:** Robust silhouette computation is subtle.; Limited to linear elliptic..

Follow-on questions.

- Extension to Robin BCs and anisotropic media?
- Variance reduction via control variates with WoS.
- Use in rendering-physics hybrid pipelines.
- Walk on Stars for PDE-constrained optimization.
- 3D performance on complex meshes.

Rationale for priority: *Beautiful paper, great code. A-tier.*

#13. Fast Poisson solvers for spectral methods

Authors: D. Fortunato;Alex Townsend • **Year:** 2017 • **Cites:** 47 • **DOI:** 10.1093/imanum/drz034

Priority: A • **Revised score:** 16/20 • **Topic:** Elliptic

Problem. Fast Poisson solvers exist for FD/FE but not for spectral methods; spectral accuracy traditionally couples with $O(n^3)$ cost.

Approach. Ultraspherical spectral discretization → Sylvester matrix equation with 'separated spectra' → ADI method gives $O(n^2 \log^2 n)$ complexity. Works on square, cylinder, solid sphere, cube.

Key results.

- 100M DOF Poisson on a square solved in ≤ 2 minutes on laptop.
- Spectral accuracy with quasi-optimal complexity.
- Integrated into Chebfun via `chebfun2.poisson`.

Main contribution. First optimal-complexity spectral Poisson solver via ADI on Sylvester equations.

Reproducibility. Code available: *yes*. URL: <https://github.com/danfortunato/fast-poisson-solvers> (HTTP 200) + Chebfun `chebop2`. Language: MATLAB (Chebfun). Completeness: 9/10. Data: Analytical test functions in repo.

Benchmarks. Poisson on square, cylinder, solid sphere, cube

Reproduction plan (feasibility 8/10, ~ 4 h, compute: CPU (single laptop)):

- 1. Install Chebfun in MATLAB/Octave.
- 2. `u = chebfun2.poisson(f, BC, n)` for $n=4096 \rightarrow$ measure time.
- 3. Verify runtime scaling across $n=64, \dots, 16384$.
- 4. Benchmark vs. MATLAB's built-in Poisson solver on same grid.

Expected fidelity: Exact match on runtime scaling; residual at machine precision.. **Known risks:** MATLAB license; Octave support for Chebfun is partial..

Follow-on questions.

- Port to Python (via `chebpy`)?
- Extension to variable-coefficient Poisson?
- Biharmonic and Helmholtz analogs via same ADI approach?
- GPU implementation?
- Use as preconditioner in more general elliptic solvers.

Rationale for priority: *Classic, clean, reproducible. MATLAB/Chebfun dependency is minor. A-tier.*

#14. Deep Reinforcement Learning for Adaptive Mesh Refinement

Authors: C. Foucart; A. Charous; Pierre FJ Lermusiaux • **Year:** 2022 • **Cites:** 35 • **DOI:** 10.48550/arXiv.2209.12351

Priority: C • **Revised score:** 11/20 • **Topic:** AMR/Multigrid

Problem. AMR refinement heuristics are ad-hoc; can RL learn better refinement policies?

Approach. Formulate AMR as POMDP with local per-element observation. Train deep Q-network to refine/coarsen HDG-FEM elements on advection and Poisson.

Key results.

- RL agent reduces DOF by ~ 20 -30% at fixed error vs heuristic.
- Generalizes across PDEs (trained on advection, tested on Poisson).
- Demonstrated on 1D & 2D.

Main contribution. First practical RL-AMR controller shown to beat bulk-refinement heuristic on DG-FEM.

Reproducibility. Code available: *no*. URL: github.com/corbett5/amr_dl (DEAD 404) | request from authors. Language: python (custom HDG code + PyTorch/RLlib). Completeness: 3/10. Data: Manufactured solutions.

Benchmarks. advection 1D/2D, Poisson 1D with Gaussian bumps

Reproduction plan (feasibility 5/10, ~ 12 h, compute: 1 GPU for RL training + CPU for DG):

- 1. Build advection HDG solver in MFEM with refinement API.

- 2. Wrap as Gym environment: obs=element residual features, action=refine/coarsen/nothing.
- 3. Train DQN per paper Section 4.
- 4. Compare agent vs bulk(theta=0.5) heuristic on Pareto.

Expected fidelity: Qualitative reproduction of Pareto dominance; quantitative within 30%.. **Known risks:** Dead code repo → full reimplementaiton.; HDG implementation details non-trivial..

Follow-on questions.

- Can the agent generalize across solver types (FVM, DG, CG)?
- Multi-agent RL for parallel AMR load-balancing.
- Use of transformer-based policy for long-range correlations.
- RL-AMR for time-dependent problems with shocks.
- Offline RL from existing AMR simulation data.

Rationale for priority: *Dead code repo. Reimplementation cost is high for what it delivers.*

#15. Godunov loss functions for modelling of hyperbolic conservation laws

Authors: R. Cassia;R. Kerswell • **Year:** 2024 • **Cites:** 5 • **DOI:** 10.1016/j.cma.2025.117782

Priority: B • **Revised score:** 13/20 • **Topic:** Hyperbolic

Problem. Standard PINNs fail for hyperbolic conservation laws with shocks because PDE residual is ill-defined at discontinuities.

Approach. Replace PDE-residual loss with finite-volume-based Godunov/approximate-Riemann-solver flux loss — 'Godunov loss'. Evaluate losses as cell-averaged flux imbalances.

Key results.

- Beats standard PINN and FVM-based (no-Riemann) losses on 2D Riemann problem.
- Successful time-stepping and super-resolution reconstruction.
- Tested across 6 classes of Riemann problems (19 admissible initial conditions).

Main contribution. Entropy-consistent shock-capturing loss function for neural PDE solvers.

Reproducibility. Code available: *no*. URL: Check CMAME supplementary; search R. Cassia or Kerswell group pages. Language: python (PyTorch). Completeness: 4/10. Data: 2D Riemann initial conditions (analytical).

Benchmarks. 2D Riemann problem (6 configuration classes)

Reproduction plan (feasibility 6/10, ~8h, compute: 1 GPU):

- 1. Reimplement Godunov loss with HLLC or Roe flux.
- 2. Generate 2D Riemann initial conditions for one representative configuration per class.
- 3. Train PINN with Godunov loss; compare to standard PDE loss.
- 4. Verify sharpness of captured shocks vs TV.

Expected fidelity: Qualitative reproduction of shock sharpness; quantitative MSE within 2×.. **Known risks:** Riemann solver implementation details.; No public code..

Follow-on questions.

- Extension to 3D Euler?
- Entropy regularization vs Godunov — ablation.
- Does WENO loss do better than Godunov loss?
- Coupling with residual-based adaptive sampling.

- Performance on ideal MHD?

Rationale for priority: *Important topic, tractable reimplementaion, but no public code. B tier.*

#16. Plane Wave Discontinuous Galerkin Methods for the 2D Helmholtz Equation: Analysis of the p-Version

Authors: R. Hiptmair; A. Moiola; I. Perugia • **Year:** 2011 • **Cites:** 199 • **DOI:** 10.1137/090761057

Priority: C • **Revised score:** 11/20 • **Topic:** Helmholtz/Wave

Problem. Plane-wave DG methods (Trefftz type) for Helmholtz equation lacked sharp p-version convergence theory.

Approach. Prove p-version convergence estimates for plane-wave DG on 2D Helmholtz; builds on ultra-weak variational formulation.

Key results.

- Algebraic convergence rate in p with explicit constants.
- Connection to standard p-FEM theory.
- Numerical experiments confirming theoretical rates.

Main contribution. Rigorous p-version analysis of plane-wave DG for Helmholtz.

Reproducibility. Code available: *no*. URL: Not advertised; Hiptmair group ETH may have MATLAB code.. Language: MATLAB likely. Completeness: 3/10. Data: Analytical Helmholtz solutions on polygonal domains.

Benchmarks. 2D Helmholtz on square with Robin BCs

Reproduction plan (feasibility 5/10, ~10h, compute: CPU):

- 1. Implement plane-wave DG basis in NGSolve.
- 2. Solve 2D Helmholtz on square for $k=1,5,10$; vary p.
- 3. Measure L2 error vs analytical solution; plot convergence in p.
- 4. Compare observed rate to theory.

Expected fidelity: Quantitative match on p-convergence rate.. **Known risks:** Plane-wave basis has conditioning issues for large p.; No public code..

Follow-on questions.

- Extension to 3D Helmholtz?
- Preconditioning of plane-wave DG systems.
- Coupling with high-order BEM.
- Time-harmonic elasticity generalization.
- High-frequency asymptotics connection.

Rationale for priority: *Theory-heavy, no code, specialized niche. Demote.*

#17. Galerkin Approximations for the Stochastic Burgers Equation

Authors: D. Blömker; Arnulf Jentzen • **Year:** 2013 • **Cites:** 105 • **DOI:** 10.1137/110845756

Priority: C • **Revised score:** 10/20 • **Topic:** NonlinearScalar

Problem. Rigorous L convergence rates for Galerkin approximations of stochastic Burgers & related SPDEs.

Approach. Prove spectral Galerkin approximations converge in L (not just L2) for semilinear stochastic evolution equations with additive noise. Apply to stochastic heat, reaction-diffusion, Burgers.

Key results.

- Explicit rate $N^{-\frac{1}{2}}$ in truncation with tied to noise regularity.
- L error uniformly in space (new — previously only L2 known).
- Numerical experiments illustrating rates.

Main contribution. First L error estimates for Galerkin-SPDE approximations.

Reproducibility. Code available: *no*. URL: **No code advertised**. Language: MATLAB likely. Completeness: 2/10. Data: Manufactured SPDE solutions.

Benchmarks. Stochastic heat, Stochastic reaction-diffusion, Stochastic Burgers

Reproduction plan (feasibility 5/10, ~8h, compute: CPU):

- 1. Implement Fourier Galerkin solver for stochastic Burgers.
- 2. Use Gaussian white-noise Q-Wiener forcing.
- 3. Monte Carlo over 100+ paths; measure L error.
- 4. Verify empirical rate matches theoretical .

Expected fidelity: Quantitative rate match.. **Known risks:** Monte Carlo variance; need many paths..

Follow-on questions.

- Sharpness of constants?
- Extension to multiplicative noise.
- L rates for finite-element vs spectral.
- Use in uncertainty quantification.
- Stochastic Burgers with shocks.

Rationale for priority: *Pure-math paper; easy to reimplement numerics but limited 'reproduction' since no code. C tier.*

#18. FLUPS: A Fourier-Based Library of Unbounded Poisson Solvers

Authors: Denis-Gabriel Caprace; Thomas Gillis; P. Chatelain • **Year:** 2020 • **Cites:** 18 • **DOI:** 10.1137/19M1303848
Priority: B • **Revised score:** 14/20 • **Topic:** Elliptic

Problem. Fast Poisson solvers needed for simulations with mixed boundary types (periodic, Neumann, unbounded), particularly vortex methods.

Approach. FLUPS: MPI+OpenMP C++ library, FFT-based Green's function convolution, spectral to order 10; supports all 3x3x3=27 BC combinations. Optimized all-to-all and point-to-point communications.

Key results.

- Convergence orders 2 to 10+ achieved.
- Strong scaling to 32K cores on unbounded Poisson.
- Faster than PMG/FMM at same accuracy for cell-centered unbounded problems.

Main contribution. Production-grade MPI Poisson library handling the full BC matrix.

Reproducibility. Code available: *yes*. URL: https://gitlab.uliege.be/uliege_mmech/flups (HTTP 200). Language: C++ (MPI + OpenMP, FFTW). Completeness: 9/10. Data: Analytical Poisson test problems in repo.

Benchmarks. Poisson 3D with periodic/Neumann/unbounded BCs, vortex ring

Reproduction plan (feasibility 7/10, ~8h, compute: multi-CPU node; HPC for scaling test):

- 1. Clone flups; build with FFTW.
- 2. Run convergence test on 3D Poisson; verify order 2,4,6,8,10.

- 3. Strong-scaling test on 128-1024 cores.
- 4. Compare timing with PETSc's FFT Poisson.

Expected fidelity: Exact match on convergence orders; qualitative match on scaling.. **Known risks:** FFTW MPI setup.; Access to HPC queue for scaling study..

Follow-on questions.

- GPU port using cuFFT / heFFTe?
- Integration with a vortex particle method.
- Extension to non-uniform grids?
- Helmholtz variant.
- Benchmark vs AccFFT.

Rationale for priority: *Strong code but HPC-oriented; needs multi-node to really show off. B tier.*

#19. Kernel-based active subspaces with application to computational fluid dynamics parametric problems using the discontinuous Galerkin method

Authors: F. Romor;M. Tezzele;Andrea Lario;G. Rozza • **Year:** 2020 • **Cites:** 15 • **DOI:** 10.1002/nme.7099
Priority: B • **Revised score:** 13/20 • **Topic:** Hyperbolic

Problem. Active subspaces reduce parameter space linearly; most engineering problems have nonlinear active structures.

Approach. Kernel-based Active Subspaces (KAS) generalize AS with feature maps (Random Fourier Features, radial kernels) and multivariate output objectives. Pipeline with DG-FEM CFD.

Key results.

- KAS outperforms linear AS on test functions with nonlinear ridge structure.
- Applied to DG-FEM CFD parametric problem (airfoil).
- Response-surface accuracy better than Gaussian process baselines.

Main contribution. Kernel-AS method for nonlinear dimension reduction in PDE-parametric problems.

Reproducibility. Code available: *yes*. URL: <https://github.com/mathLab/EZyRB> (HTTP 200) + ATHENA from same group. Language: python. Completeness: 7/10. Data: Benchmark test functions + HopeFOAM CFD runs.

Benchmarks. Radial test functions, SEIR model, DG-FEM airfoil

Reproduction plan (feasibility 7/10, ~6h, compute: CPU):

- 1. Install ATHENA from mathLab.
- 2. Run radial test function experiments; reproduce linear-vs-kernel AS comparison.
- 3. (Optional) Run HopeFOAM airfoil case for CFD pipeline.

Expected fidelity: Quantitative R^2 match on test functions.. **Known risks:** HopeFOAM case requires DG solver stack.; Random seed affects RFF variance..

Follow-on questions.

- Deep-kernel AS with learned features?
- Use in Bayesian optimization for expensive PDE surrogates.
- Extension to multi-fidelity setups.
- Automatic kernel selection.
- Active learning of subspace directions.

Rationale for priority: *Tool-chain heavier than advertised; fine B-tier candidate for UQ/surrogates.*

#20. On the convergence of discontinuous Galerkin/Hermite spectral methods for the Vlasov-Poisson system

Authors: M. Bessemoulin-Chatard; F. Filbet • **Year:** 2022 • **Cites:** 6 • **DOI:** 10.48550/arXiv.2208.12503
Priority: C • **Revised score:** 10/20 • **Topic:** Elliptic

Problem. DG/Hermite-spectral methods for Vlasov-Poisson lacked a priori convergence theory.

Approach. Use asymmetrically weighted Hermite in velocity + DG in space. Introduce time-dependent weighted L2 space. Prove stability + error estimate.

Key results.

- Global weighted L2 stability.
- Propagation of regularity.
- Error estimate in weighted L2.

Main contribution. First convergence theory for DG/Hermite Vlasov-Poisson.

Reproducibility. Code available: *no*. URL: **No code advertised**. Language: Julia/Fortran likely. Completeness: 2/10. Data: Manufactured Vlasov-Poisson solutions.

Benchmarks. Landau damping, two-stream instability, bump-on-tail

Reproduction plan (feasibility 4/10, ~12h, compute: CPU):

- 1. Implement DG-Hermite Vlasov-Poisson from scratch in Python + FEniCSx.
- 2. Landau damping test; verify damping rate matches linear theory.
- 3. Refine Hermite moments; measure convergence.
- 4. Compare to reported rate.

Expected fidelity: Qualitative match; quantitative rate within 20%.. **Known risks:** Significant implementation effort; weighted-Hermite basis conditioning..

Follow-on questions.

- Extension to Vlasov-Maxwell?
- Positivity-preserving DG-Hermite schemes.
- Kinetic plasma turbulence benchmarks.
- Comparison to semi-Lagrangian.
- GPU port.

Rationale for priority: *Theory paper; heavy reimplementations, no code. C tier.*

#21. Improvements to the APBS biomolecular solvation software suite

Authors: E. Jurrus; David Engel; Keith T. Star; Kyle Monson; Juan Brandi; Lisa E. Felberg; David H. Brookes; Leighton Wilson; Jiahui Chen; Karina Liles; Minju Chun; Peter Li; D. Gohara; Todd J. Dolinsky; R. Konečný; D. Koes; •
Year: 2017 • **Cites:** 2081 • **DOI:** 10.1002/pro.3280

Priority: A • **Revised score:** 15/20 • **Topic:** Elliptic

Problem. Update and extend APBS for biomolecular electrostatics (Poisson-Boltzmann equation).

Approach. New features in APBS suite: analytical/semi-analytical PBE solver, boundary element solver, geometric flow solvation, graph-based pKa, visualization.

Key results.

- Scalable to large biomolecules.

- Multiple solver backends (FD multigrid, BEM, analytical).
- Validated on standard benchmark sets.

Main contribution. Reference biomolecular electrostatics toolkit, widely used (2081 cites).

Reproducibility. Code available: *yes*. URL: <https://github.com/Electrostatics/apbs> (HTTP 200). Language: C/C++ with Python bindings. Completeness: 10/10. Data: PQR files from PDB, provided in docs.

Benchmarks. alpha-chymotrypsin, insulin hexamer, lysozyme, 10+ standard biomol cases

Reproduction plan (feasibility 9/10, ~6h, compute: CPU (multi-core)):

- 1. conda install -c conda-forge apbs pdb2pqr.
- 2. Run example inputs from APBS tutorial (1ake lysozyme etc).
- 3. Reproduce solvation energy values.
- 4. Use geometric-flow solver on one test; compare to FD-multigrid.

Expected fidelity: Quantitative match to within 1-2%.. **Known risks:** Compiling APBS from source is painful; conda package is stable..

Follow-on questions.

- Integration with ML potentials (FEP/ML).
- Coupling with MD via AMBER PBSA.
- GPU port of geometric flow.
- AlphaFold → APBS automated pipeline.
- pKa prediction benchmark vs PROPKA.

Rationale for priority: *Mature code, easy conda install. Upgrade — a very safe starter replication.*

#22. Fast Nonsymmetric Iterations and Preconditioning for Navier-Stokes Equations

Authors: H. Elman;D. Silvester • **Year:** 1996 • **Cites:** 258 • **DOI:** 10.1137/0917004

Priority: B • **Revised score:** 14/20 • **Topic:** Fluid/NavierStokes

Problem. Classic Navier-Stokes linear systems are non-symmetric + indefinite; few preconditioners give mesh-independent convergence.

Approach. Block preconditioners for discrete Oseen operator: diagonal with Schur-complement approximations. Prove eigenvalues bounded independent of h.

Key results.

- GMRES iterations independent of mesh size h.
- Inner iterative solvers for preconditioner keep costs low.
- Validated on backward-facing step and lid-driven cavity.

Main contribution. Foundational paper (258 cites) for block preconditioning of incompressible Navier-Stokes — precursor to PCD/LSC.

Reproducibility. Code available: *no*. URL: **None advertised (1996 paper)**; modern equivalents in IFISS (MATLAB). Language: MATLAB. Completeness: 6/10. Data: Standard Stokes/Oseen benchmarks.

Benchmarks. Backward-facing step, lid-driven cavity, channel flow

Reproduction plan (feasibility 8/10, ~5h, compute: CPU):

- 1. Download IFISS; run `navier.testproblem` with the block preconditioner.
- 2. Reproduce iteration-vs-h table at multiple Re.
- 3. Verify mesh-independence claim.

Expected fidelity: Quantitative iteration-count reproduction.. **Known risks:** MATLAB license for IFISS.; Original 1996 numbers may differ slightly from IFISS defaults..

Follow-on questions.

- Compare to LSC and PCD preconditioners.
- Extend to stabilized FE discretizations.
- Use in ROM surrogates.
- GPU port of block preconditioner.
- Extension to Navier-Stokes-Boussinesq.

Rationale for priority: *Classic, reproducible via IFISS. Solid B.*

#23. New Laplace and Helmholtz solvers

Authors: A. Gopal;L. Trefethen • **Year:** 2019 • **Cites:** 51 • **DOI:** 10.1073/pnas.1904139116

Priority: A • **Revised score:** 15/20 • **Topic:** Helmholtz/Wave

Problem. Standard FEM/BEM methods have $O(h^p)$ polynomial convergence on polygons with corners.

Approach. Rational-function approximations (lightning solvers) with exponential clustering of poles near corners give fractional-exponential (root-exponential) convergence for 2D Laplace/Helmholtz.

Key results.

- e^{-cN} convergence for 2D Laplace on polygons with corners.
- Orders of magnitude faster than FEM/BEM for comparable accuracy.
- Implemented in lightning.m (MATLAB).

Main contribution. Paradigm shift toward rational-function PDE solvers (lightning Laplace solver).

Reproducibility. Code available: *yes*. URL: <https://people.maths.ox.ac.uk/trefethen/lightning.html> (MATLAB scripts). Language: MATLAB. Completeness: 9/10. Data: Analytical Laplace problems on L-shape, polygons.

Benchmarks. Laplace on L-shape, Helmholtz on polygon with corners

Reproduction plan (feasibility 9/10, ~3h, compute: CPU (laptop)):

1. Download lightning.m from Trefethen's page.
2. Run L-shape example; verify 10-digit accuracy.
3. Vary N ; plot convergence vs e^{-cN} .
4. Compare time-to-10-digits vs FEniCS on same geometry.

Expected fidelity: Exact match at machine precision.. **Known risks:** MATLAB license..

Follow-on questions.

- Python port (already exists via Baddoo's lightning-python)?
- 3D extension?
- Lightning Helmholtz at high k .
- Combining lightning with FEM (hybrid)?
- Use in conformal mapping.

Rationale for priority: *Short, clean, high-impact paper with a one-file MATLAB demo. Perfect reproduction target.*

#24. Stability and error analysis of a class of high-order IMEX schemes for Navier-stokes equations with periodic boundary conditions

Authors: Fukeng Huang;Jie Shen • **Year:** 2021 • **Cites:** 43 • **DOI:** 10.1137/21m1404144

Priority: B • **Revised score:** 12/20 • **Topic:** Fluid/NavierStokes

Problem. High-order IMEX schemes for Navier-Stokes lacked unconditional-stability error analysis.

Approach. Scalar-auxiliary-variable (SAV) technique + IMEX k-step ($k=1..5$) + Fourier-Galerkin space. Prove unconditional L^2 stability and error estimates.

Key results.

- First unconditionally stable high-order IMEX for Navier-Stokes.
- Up to 5th-order in time, confirmed numerically.
- Successful on double-shear-layer problem.

Main contribution. Rigorous high-order IMEX-SAV framework for incompressible NS.

Reproducibility. Code available: *no*. URL: Shen group sometimes publishes MATLAB on personal pages | check. Language: MATLAB likely. Completeness: 3/10. Data: Manufactured solutions + double shear layer IC.

Benchmarks. Taylor-Green 2D, double shear layer 2D

Reproduction plan (feasibility 6/10, ~8h, compute: CPU):

- 1. Implement Fourier pseudo-spectral Navier-Stokes in 2D.
- 2. Add SAV auxiliary variable; implement IMEX-BDFk for $k=1..5$.
- 3. Manufactured-solution test; verify order k in t .
- 4. Double shear layer at $Re=10000$; compare vorticity to reported figure.

Expected fidelity: Quantitative order match; qualitative vorticity match.. **Known risks:** SAV implementation subtleties.; High-order IMEX conditioning..

Follow-on questions.

- SAV + exponential integrators?
- Extension to MHD.
- SAV on unstructured FE meshes.
- Combination with adaptive time-stepping.
- Parallelization.

Rationale for priority: *Clear numerics but no code; reimplementing tractable.*

#25. Rate optimality of adaptive finite element methods with respect to the overall computational costs

Authors: G. Gantner;A. Haberl;D. Praetorius;S. Schimanko • **Year:** 2020 • **Cites:** 34 • **DOI:** 10.1090/mcom/3654

Priority: C • **Revised score:** 11/20 • **Topic:** Elliptic

Problem. Adaptive FEM rate-optimality theory typically measured vs # DOF, not vs computational cost.

Approach. Analyze adaptive FEM with inexact iterative solvers (CG for linear, Zarantonello for nonlinear monotone). Prove quasi-optimal linear convergence w.r.t. total cost.

Key results.

- Rate-optimality in overall cost (not just DOF).
- Applies to linear elliptic and nonlinear monotone problems.
- Numerical experiments confirm.

Main contribution. First rate-optimality theorem w.r.t. overall computational cost for inexact AFEM.

Reproducibility. Code available: *no*. URL: TU Wien group sometimes publishes MATLAB. Language: MATLAB likely. Completeness: 3/10. Data: Standard elliptic test problems.

Benchmarks. L-shape Laplace, Z-shape elliptic, p-Laplacian nonlinear

Reproduction plan (feasibility 6/10, ~8h, compute: CPU):

- 1. Implement residual-based AFEM in FEniCS with Dörfler marking.
- 2. Use inexact CG for linear system; adjust solver tolerance adaptively.
- 3. Track error vs cumulative solver cost on L-shape.
- 4. Confirm algebraic rate -1 w.r.t. cost.

Expected fidelity: Qualitative reproduction of rate.. **Known risks:** AFEM implementation details subtle.; No public reference code..

Follow-on questions.

- Extension to parabolic problems.
- Goal-oriented AFEM cost-optimality.
- Practical comparison w/ heuristic AFEM.
- Multigrid inside inexact solver.
- ML-based marking strategies.

Rationale for priority: *Theory-focused, no code. C.*

#26. Comparison of Adaptive Multiresolution and Adaptive Mesh Refinement Applied to Simulations of the Compressible Euler Equations

Authors: R. Deiterding; M. O. Domingues; S. Gomes; K. Schneider • **Year:** 2015 • **Cites:** 33 • **DOI:** 10.1137/15M1026043

Priority: B • **Revised score:** 13/20 • **Topic:** AMR/Multigrid

Problem. AMR and adaptive-multiresolution (MR) are two dominant approaches for PDEs with sharp fronts; which is better?

Approach. Head-to-head FV comparison on compressible Euler (2D Riemann Lax-Liu #6, 3D expanding shock). Both use 2nd-order shock-capturing + local time-stepping.

Key results.

- MR has better memory compression and slightly enhanced convergence.
- AMR has lower absolute overhead in tested codes.
- CPU-time compression trends similar for both.

Main contribution. Quantitative comparison settling MR vs AMR tradeoffs on standard Euler benchmarks.

Reproducibility. Code available: *yes*. URL: <https://github.com/waveletApplications/carmen> (HTTP 200) for MR side; AMROC for AMR. Language: C/C++/Fortran. Completeness: 7/10. Data: Riemann IC analytical.

Benchmarks. 2D Riemann Lax-Liu #6, 3D expanding shock ellipsoid

Reproduction plan (feasibility 7/10, ~8h, compute: multi-CPU):

- 1. Build Carmen; run Lax-Liu #6.
- 2. Build AMROC; run same problem.
- 3. Compare memory and CPU vs regular-grid baseline.
- 4. Confirm MR memory advantage.

Expected fidelity: Quantitative compression factor match within 20%.. **Known risks:** AMROC build is specialized.; Timings hardware-dependent..

Follow-on questions.

- Hybrid MR+AMR?
- Performance on 3D turbulence.
- GPU AMR frameworks (AMReX) comparison.
- Wavelet-based refinement indicator portability.
- Extension to relativistic hydro.

Rationale for priority: *Both codes exist but heavy-weight; mid-tier B.*

#27. Kinetic.jl: A portable finite volume toolbox for scientific and neural computing

Authors: Tianbai Xiao • **Year:** 2021 • **Cites:** 17 • **DOI:** 10.21105/joss.03060

Priority: A • **Revised score:** 15/20 • **Topic:** ML-operator/PINN

Problem. Need a flexible FV framework for kinetic equations + scientific ML in Julia.

Approach. Kinetic.jl: modular FV toolbox with KitBase (physics) and KitML (neural closures). Supports multi-scale Boltzmann-Navier-Stokes transitions.

Key results.

- Solves Navier-Stokes and Boltzmann in unified API.
- Neural closure example for BGK collision operator.
- Short JOSS paper (software publication).

Main contribution. Unified kinetic-theory + ML Julia toolbox.

Reproducibility. Code available: *yes*. URL: <https://github.com/vavrines/Kinetic.jl> (HTTP 200). Language: Julia. Completeness: 9/10. Data: Test scripts in repo.

Benchmarks. Sod shock tube, Couette flow, BGK Boltzmann

Reproduction plan (feasibility 9/10, ~4h, compute: CPU):

1. Install Julia 1.10+; add Kinetic.jl.
2. Run examples/sod.jl; verify shock tube matches analytical Riemann solution.
3. Run Couette flow.
4. Reproduce a neural-closure example from KitML.

Expected fidelity: Quantitative match on shock positions/states.. **Known risks:** Julia version compatibility with older package commits..

Follow-on questions.

- Multi-scale coupling between kinetic and NS regimes.
- GPU support via CUDA.jl.
- Plasma kinetics.
- Neural BGK operator learning benchmarks.
- Adaptive grid refinement inside Kinetic.jl.

Rationale for priority: *Julia ease + short paper = very low friction A-tier replication.*

#28. Highly efficient and energy dissipative schemes for the time fractional Allen-Cahn equation

Authors: Dianming Hou;Chuanju Xu • **Year:** 2021 • **Cites:** 17 • **DOI:** 10.1137/20m135577x

Priority: C • **Revised score:** 10/20 • **Topic:** PhaseField/RD

Problem. Time-fractional Allen-Cahn equation needs stable schemes on graded meshes for efficient long-time simulation.

Approach. Split fractional derivative into local + history. Use L1/L1-CN/L1+-CN for local; extended SAV for nonlinearity + history. Fourier-Galerkin space.

Key results.

- Unconditional energy dissipation proven.
- Dissipation holds on graded mesh.
- Numerical confirmation on 2D Allen-Cahn.

Main contribution. First unconditionally dissipative high-order scheme for time-fractional Allen-Cahn.

Reproducibility. Code available: *no*. URL: **No code advertised**. Language: MATLAB likely. Completeness: 2/10. Data: Manufactured + standard Allen-Cahn IC.

Benchmarks. Time-fractional Allen-Cahn 2D

Reproduction plan (feasibility 5/10, ~10h, compute: CPU):

- 1. Implement L1+-CN on graded mesh with sum-of-exponentials history.
- 2. Manufactured solution test for convergence order.
- 3. 2D Allen-Cahn coarsening; verify energy monotonicity.

Expected fidelity: Quantitative order match.. **Known risks:** Sum-of-exponentials approximation accuracy.; No reference code..

Follow-on questions.

- Extension to time-fractional Cahn-Hilliard.
- Higher-order L2/L3 schemes.
- Parallel SoE history evaluation.
- Coupling to phase-field crystal models.
- Adaptive time-stepping.

Rationale for priority: *No code; specialized.*

#29. Modified Poisson-Nernst-Planck Model with Coulomb and Hard-sphere Correlations

Authors: Manman Ma;Zhenli Xu;Liwei Zhang • **Year:** 2020 • **Cites:** 15 • **DOI:** 10.1137/19m1310098

Priority: C • **Revised score:** 10/20 • **Topic:** Elliptic

Problem. Standard Poisson-Nernst-Planck misses Coulomb and hard-sphere correlations important for multi-valent ions at interfaces.

Approach. Add Coulomb correlation via generalized Debye-Hückel (self-Green's function) and hard-sphere correlation via modified fundamental measure theory. Solve via WKB asymptotics for parallel-plate geometry.

Key results.

- Reproduces charge-inversion effects seen in Monte Carlo simulations.
- Agreement with MC for multivalent ions.
- Analytical WKB solution for two-plate case.

Main contribution. Modified PNP with both long- and short-range corrections beyond mean-field.

Reproducibility. Code available: *no*. URL: **No code advertised**. Language: MATLAB likely. Completeness: 2/10. Data: Reference MC data from cited papers.

Benchmarks. Parallel-plate electrolyte with multivalent ions

Reproduction plan (feasibility 5/10, ~10h, compute: CPU):

- 1. Implement generalized Debye-Hückel equation solver (1D).
- 2. Add MFMT hard-sphere correction.
- 3. WKB asymptotic solution for two plates.
- 4. Compare density profiles to reference MC data.

Expected fidelity: Qualitative match on charge inversion.. **Known risks:** Implementation of self-Green's-function iteration.; No reference code..

Follow-on questions.

- Extension to 3D finite systems.
- Coupling with ML for closure.
- Nonequilibrium transport predictions.
- Nanopore applications.
- Benchmark vs classical DFT of ions.

Rationale for priority: *Niche physics; no code*.

#30. Optimized Schwarz Methods without Overlap for the Helmholtz Equation

Authors: M. Gander;F. Magoulès;F. Nataf • **Year:** 2002 • **Cites:** 403 • **DOI:** 10.1137/S1064827501387012
Priority: B • **Revised score:** 13/20 • **Topic:** Helmholtz/Wave

Problem. Classical Schwarz for Helmholtz fails without overlap (propagating modes undamped).

Approach. Derive optimized Robin-type transmission conditions (symbol (k) optimized in Fourier). Proves convergence without overlap for Helmholtz.

Key results.

- Optimal transmission condition gives convergence in finite steps (nonlocal) + local approximation.
- Dramatic iteration-count reduction over classical Schwarz.
- Numerical validation on 2D Helmholtz benchmark.

Main contribution. Foundational (403 cites) optimized Schwarz theory for Helmholtz.

Reproducibility. Code available: *no*. URL: **None advertised (2002 paper)**; modern HPDDM library includes OSM. Language: MATLAB / FreeFem++. Completeness: 6/10. Data: Analytical Helmholtz solutions.

Benchmarks. 2D Helmholtz in rectangle

Reproduction plan (feasibility 7/10, ~5h, compute: CPU):

- 1. Implement Schwarz iteration with classical vs optimized Robin BC in FreeFem++.
- 2. 2D Helmholtz on $[0,1]^2$ at $k=10$.
- 3. Compare iteration counts.
- 4. Reproduce convergence-rate curves from paper.

Expected fidelity: Quantitative iteration-count reproduction.. **Known risks:** Tuning of parameter.; 2002 numerics environment..

Follow-on questions.

- Extension to Maxwell’s equations.
- Hybrid OSM + sweeping preconditioner.
- OSM on unstructured meshes.
- Performance at very high k .
- ML-tuning of optimized parameters.

Rationale for priority: *Classic paper reproducible via HPDDM or direct FreeFem++ implementation.*

3. Recommended 15-paper batch schedule

See companion `top30_batch_schedule.md` for details. Summary:

Wave 1 (5 parallel, ~6–8h wallclock, CPU+GPU mix): FEM-vs-PINNs, Walk on Stars, Fast Poisson Spectral, Lightning Laplace, Kinetic.jl.

Wave 2 (5 parallel, ~6–10h, 1 GPU each): `jax-cfd` (ML-CFD), Latent Spectral Models, Koopman Neural Operator, Laplace Neural Operator, Dedalus.

Wave 3 (5 parallel, mostly CPU-heavy): APBS, `lifex-cfd`, Elman-Silvester (IFISS), Optimized Schwarz, FLUPS. Total wallclock with 5-way parallelism: ~24–30 hours.

4. Conclusion

Rick: our highest-yield near-term set is the 11 Priority A papers; starting Wave 1 of 5 today produces verifiable results in a workday. Stage-5 demoted 5 papers to C due to dead/absent code; Stage-4 signal heuristics (which scored `github:...` substring presence without HTTP validation) were too optimistic on those. One 404 (Foucart 2022 RL-AMR) is a clean red flag we couldn’t have caught without this stage.